

Analysing Algorithm Runtime

Big O, Big Ω , Big Θ , and $\sim$ notation

Comparing algorithms

- We saw with union-find that there were many different valid algorithms that all solved the problem – but some of them were much, much, *much* faster than others when used on large datasets.
- We often have more than one algorithm for the same task: several ways to sort a list, or to search for an item in it.
- All of them give the right answer. What differs is how much work each one does.
- On small inputs, the difference is usually invisible: sorting 100 items takes almost no time with any method. On 10 million items, one method might finish in under a second while another takes hours.
- This slideset discusses mathematical tools we can use to formally compare algorithms

Why not just time it?

- One way to measure an algorithm is to run it and time it. Sometimes that is the right approach.
- But the measured time depends on more than the algorithm:
 - the machine it runs on
 - what else the machine is doing at the same time
 - which input you happened to test
 - the language, compiler, and runtime
- We want a description of the algorithm itself, and ideally one we can work out before writing any code.

Count the work instead

Choose the operation that runs most often, and count how many times it runs.

```
1  int max = a[0];  
2  for (int i = 1; i < n; i++) {  
3      if (a[i] > max) max = a[i];  
4  }
```

array length n	comparisons
10	9
1,000	999
1,000,000	999,999

The comparison $a[i] > \text{max}$ runs $n - 1$ times. If the array doubles in length, the number of comparisons roughly doubles.

Work that grows in proportion to n is called *linear*, and is written $O(n)$.



(this is pronounced
"big O of n")

Nested vs. sequential loops

```
1  for (int i = 0; i < n; i++) {  
2      for (int j = 0; j < n; j++) {  
3          if (a[i] == a[j]) count++;  
4      }  
5  }
```

The inner loop runs n times for each of the n passes of the outer loop.

$n \times n = n^2$ comparisons. This is *quadratic*: $O(n^2)$.

```
1  for (int i = 0; i < n; i++) { ... }  
2  
3  for (int j = 0; j < n; j++) { ... }
```

One loop runs after the other: $n + n = 2n$ steps.

This is still linear: $O(n)$. (Why the 2 can be dropped comes later.)

Rule of thumb: nested loops multiply, and sequential loops add.

Halving gives $\log n$

```
1 while (n > 1) {  
2     n = n / 2;  
3     steps++;  
4 }
```

starting n	steps to reach 1
8	3
1,024	10
1,048,576 (about a million)	20
1,073,741,824 (about a billion)	30

Each pass cuts n in half, so the loop runs about $\log_2 n$ times. Doubling n adds only one more step.

About a billion items takes only 30 steps. This is why algorithms that repeatedly halve their input, such as binary search, are so fast.

In Big-O notation, $\log_2 n$ is usually written simply as $\log n$.

Common growth rates

name	order	n = 10	n = 1,000	n = 1,000,000
logarithmic	$\log n$	3	10	20
linear	n	10	1,000	1,000,000
linearithmic	$n \log n$	33	10,000	20,000,000
quadratic	n^2	100	1,000,000	1,000,000,000,000
exponential	2^n	1,024	a 302-digit number	a 301,030-digit number

Values are rounded, and logarithms are base 2. Each row shows how one kind of algorithm's work grows as n gets larger.

At $n = 1,000,000$, n^2 is about 50,000 times larger than $n \log n$. If an $n \log n$ algorithm took one second, an n^2 algorithm would take about 14 hours.

Method of analysing runtime #1: Big-O analysis

- Big-O analysis gives an *upper bound* on how an algorithm's running time (or memory use) grows as its input gets larger.
- We can apply it to the best, average, or worst case, but we usually apply it to the *worst case*. Why?
 - It's nice to have a guarantee: running the algorithm will never take more time (or memory) than this.
 - Worst-case scenarios happen! For example, if you build a binary search tree from input that's already sorted, you end up with a linked list, losing all the advantages of a binary search tree.
 - Malicious input is a thing that exists! Eventually, someone – or some bot – running a denial-of-service attack on your server will figure out which inputs trigger the worst case.

Reading the notation: $f(n)$ is $O(g(n))$

- $f(n)$ is the exact count of operations for an input of size n . For example: $3n^2 + 12n + 7$.
- $g(n)$ is a simpler function we compare it to, such as $\log n$, n , $n \log n$, or n^2 .
- “ $f(n)$ is $O(g(n))$ ” means: once n is large enough, $f(n)$ grows no faster than $g(n)$, up to a constant multiple.
- It is an *upper bound*: a ceiling on how fast $f(n)$ can grow.

Example: $3n^2 + 12n + 7$ is $O(n^2)$.

The next slide states the same idea precisely.

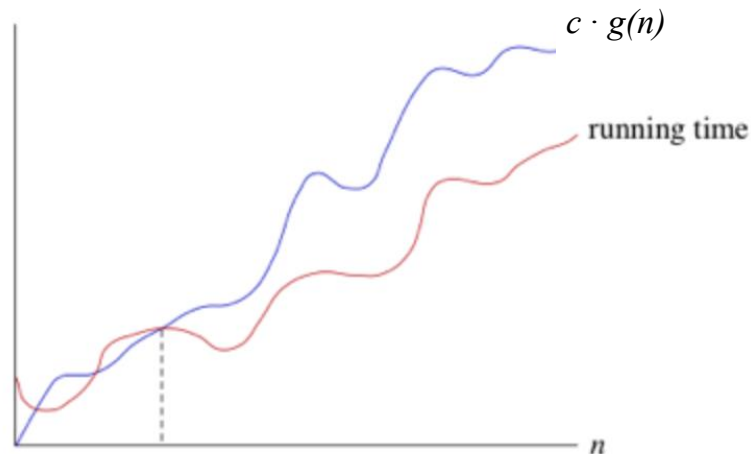
Asymptotic upper bound – formal-ish definition of big O

- We say that a function $f(n)$ is $O(g(n))$ if
 - there exist constants $c > 0$ and n_0 such that
 - $f(n) \leq c \cdot g(n)$ for all $n > n_0$
- In words: past some point n_0 , $f(n)$ never rises above some fixed multiple of $g(n)$.

Asymptotic upper bound – formal-ish definition of big O

$f(n)$ is $O(g(n))$ if and only if there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n > n_0$

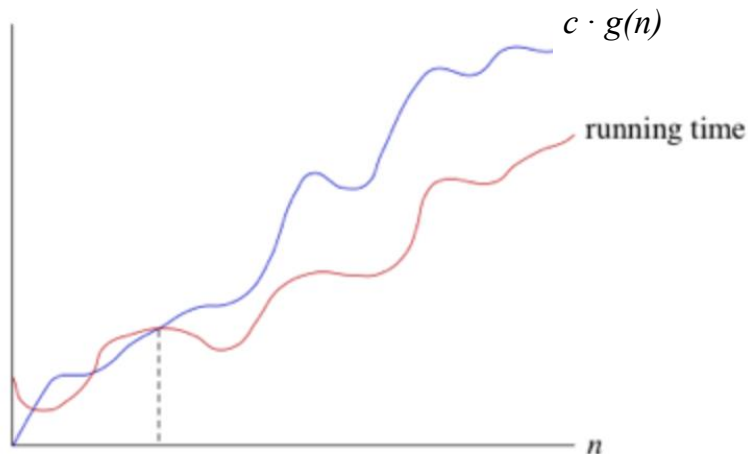
If you're anything like me, this definition was absolutely clear as mud the first time you saw it. Let's use some pictures stolen from a textbook to illustrate it.



(I stole this picture from a textbook)

Asymptotic upper bound – formal-ish definition of big O

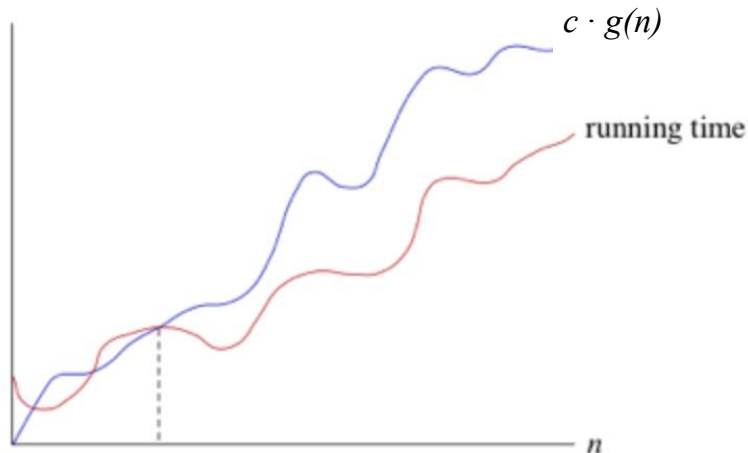
$f(n)$ is $O(g(n))$ if and only if there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n > n_0$



← The upper curve is $c \cdot g(n)$: some function $g(n)$, multiplied by some constant c . We make no assumptions about what $g(n)$ or c is.

Asymptotic upper bound – formal-ish definition of big O

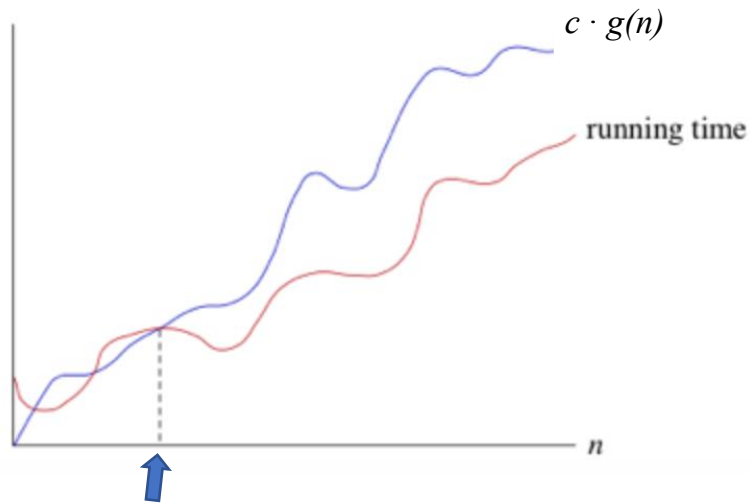
$f(n)$ is $O(g(n))$ iff there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n > n_0$



← The lower curve, $f(n)$, is the amount of time it takes to run our algorithm.

Asymptotic upper bound – formal-ish definition of big O

$f(n)$ is $O(g(n))$ iff there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n > n_0$



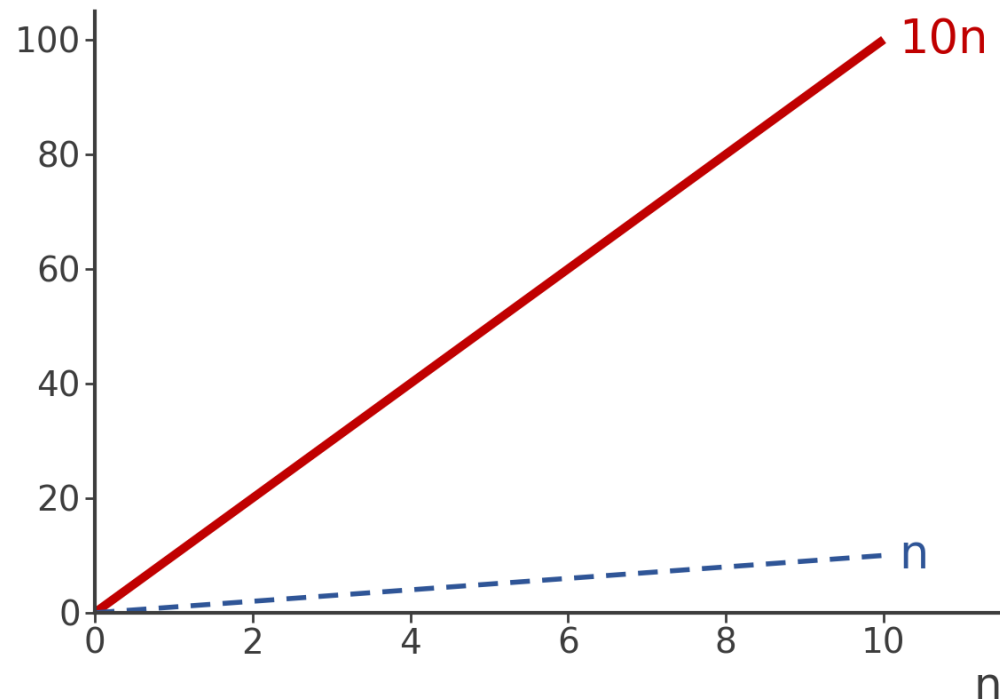
This is n_0 . We don't care about values of $n < n_0$.

When n is small, there are places where $c \cdot g(n) < f(n)$.

That's alright! We only need $f(n) \leq c \cdot g(n)$ from some point on. Any n_0 past the last crossing works; here, we picked the last crossing itself.

Asymptotic upper bound – formal-ish definition of big O

$f(n)$ is $O(g(n))$ iff there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n > n_0$



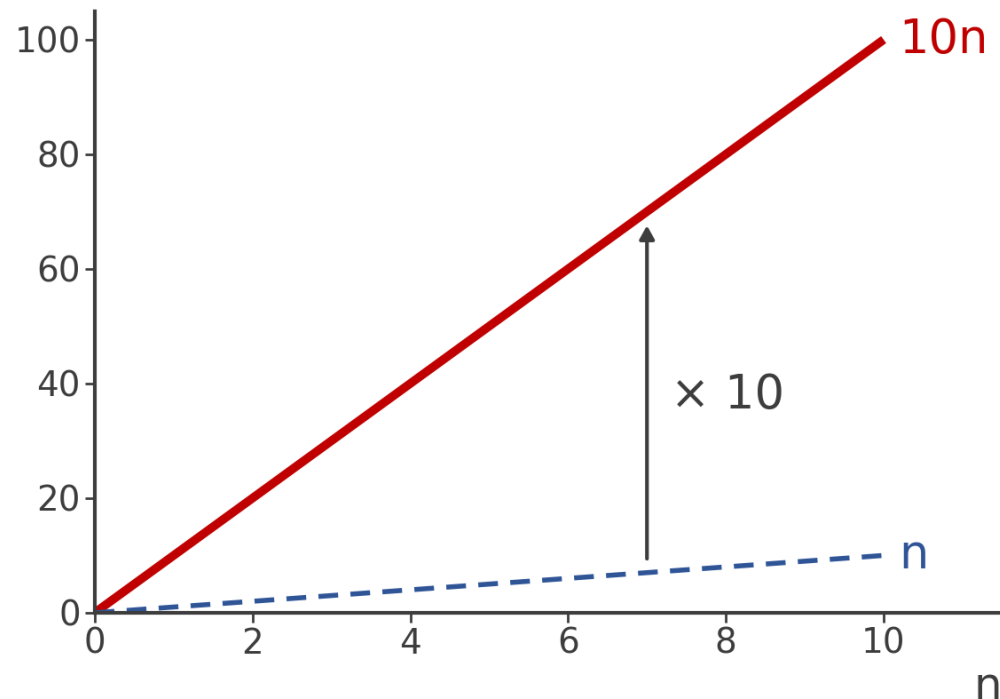
Why multiply by a constant? Let's think in terms of linear functions.

Suppose our algorithm's running time is the solid $10n$ line: it takes 10 operations for each element in the data set, so it takes $10n$ operations for a data set of n items.

Obviously, $10n > n$ for all $n > 0$... however, the running time of our algorithm is $O(n)$. Why?

Asymptotic upper bound – formal-ish definition of big O

$f(n)$ is $O(g(n))$ iff there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n > n_0$



Obviously, $10n > n$ for all $n > 0$... however, the running time of our algorithm is $O(n)$. Why?

... just set the constant c to any value ≥ 10 .
Multiply n by 10, and it's never below $10n$ again.

Why we do things this way

$f(n)$ is $O(g(n))$ iff there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n > n_0$

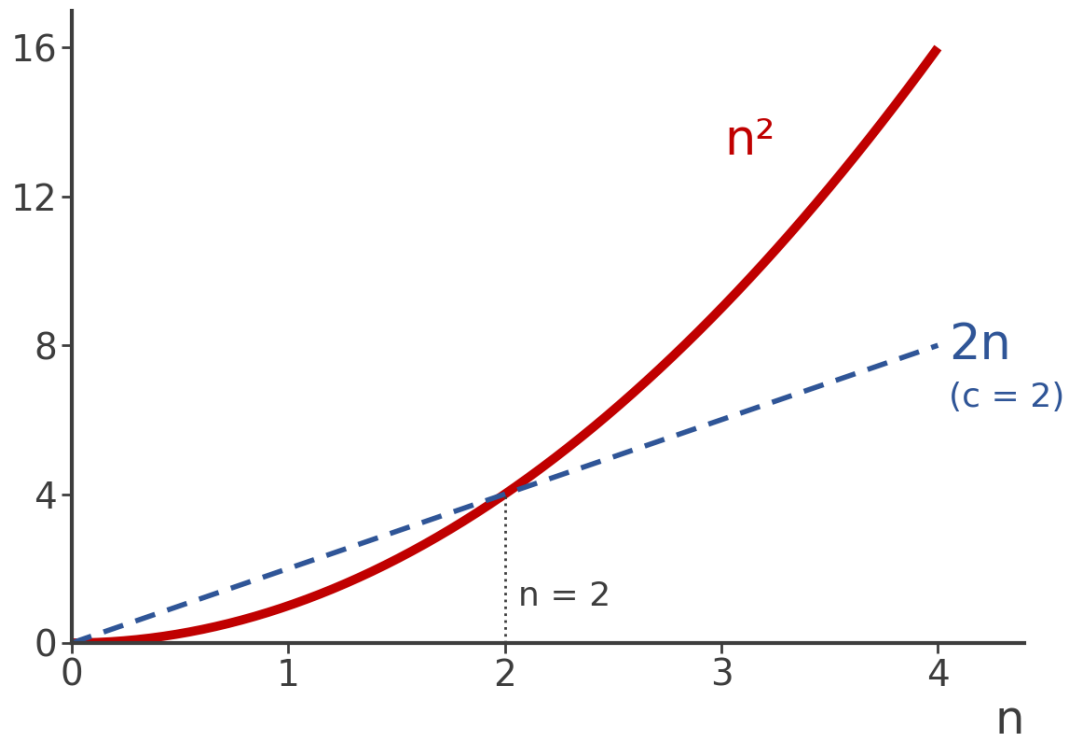
When we're analysing running times, we want to talk about them abstracted away from any particular machine the code might run on.

Improvements in (for example) computer hardware tend to give *constant-factor* speedups: a faster machine might do every operation 10 times faster, regardless of the size of n . In big-O notation, we abstract away those constant factors.

More to the point: consider the two functions on the next slide.

Why we do things this way

$f(n)$ is $O(g(n))$ iff there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n > n_0$



Let's hypothesize that n^2 is $O(n)$.

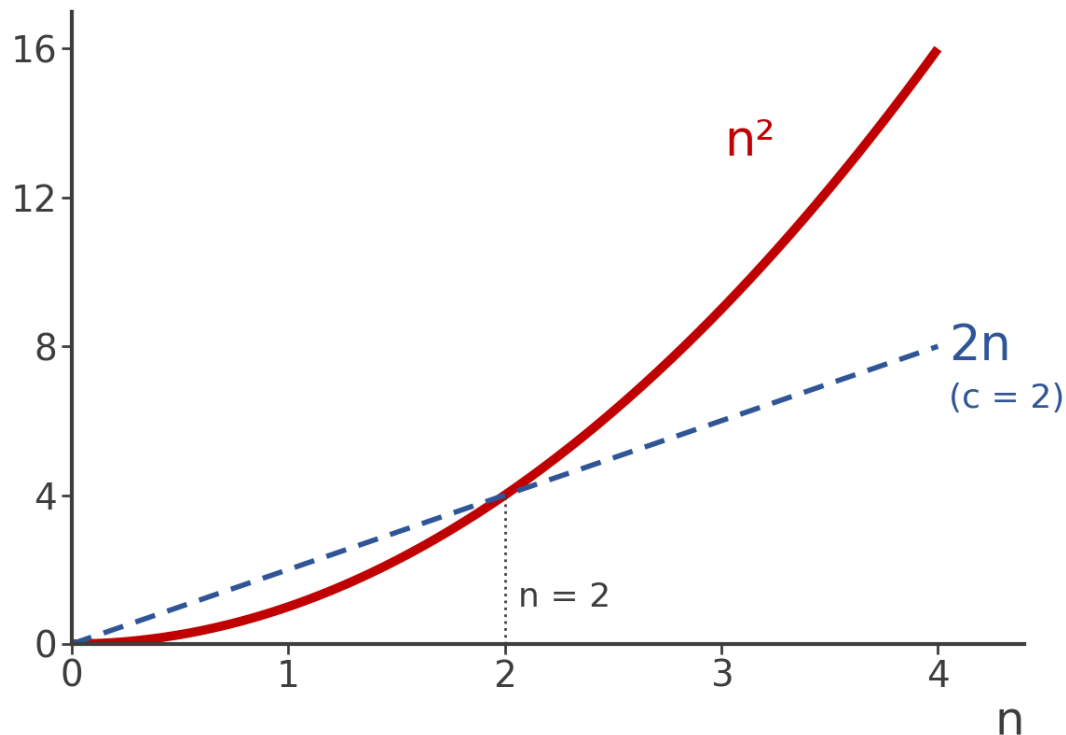
Try $c = 2$. We start out great! Until $n = 2$, $n^2 \leq 2n$.

But of course, we have to consider values of $n > 2$, too. Maybe we just need a bigger c : let's hypothesize there's some constant c such that $n^2 \leq c \cdot n$ for all large n .

We can find a c that works for any *particular* n . For example, say that n is 1,000,000. We set c to 1,000,001, and see that $1,000,000^2$ really is $\leq 1,000,000 \times 1,000,001$.

Why we do things this way

$f(n)$ is $O(g(n))$ iff there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n > n_0$



We can find a c that works for any *particular* n : for $n = 1,000,000$, $c = 1,000,001$ does it.

But $n^2 \leq c \cdot n$ is the same as $n \leq c$ (divide both sides by n). So c would have to be at least as big as *every* n past n_0 .

That's not possible: whatever c and n_0 you pick, I'll pick an n that's bigger than both.

Therefore, n^2 is **not** $O(n)$.

Why we do things this way

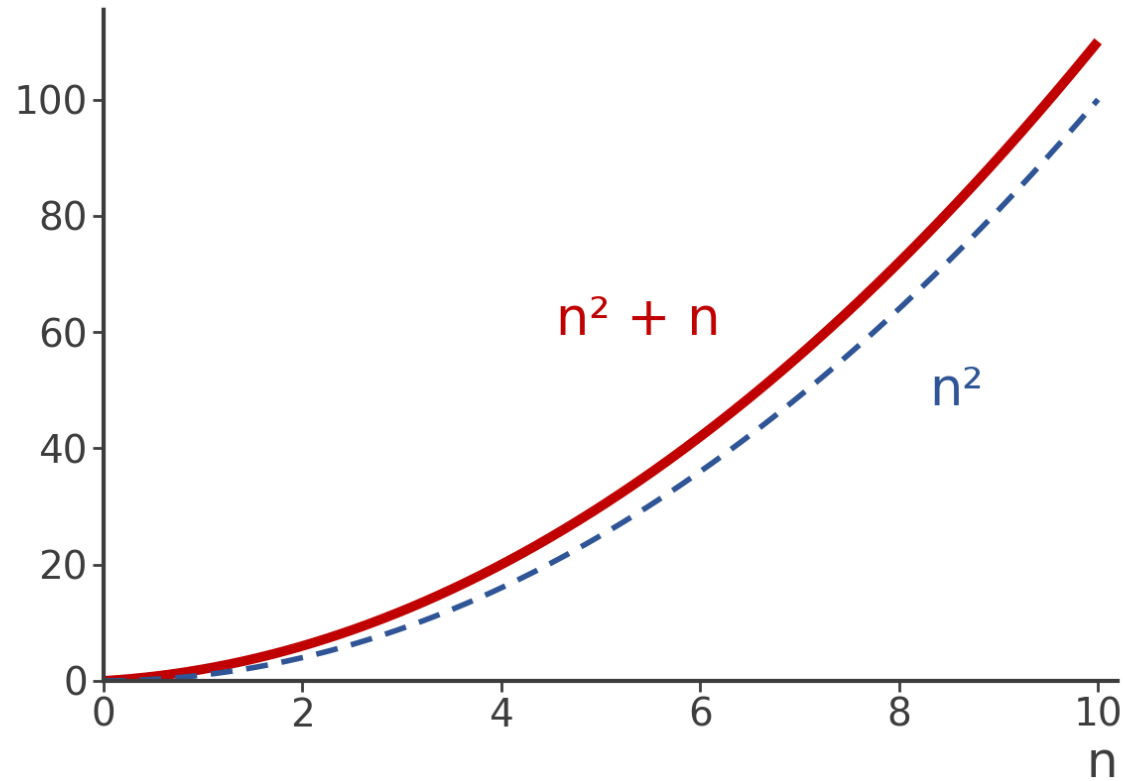
$f(n)$ is $O(g(n))$ iff there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n > n_0$

In big-O notation, we drop *constant factors*. We don't write $O(10n)$: it means the same thing as $O(n)$, because the 10 can be rolled into c . ($10n$ is $O(n)$ because $10n \leq c \cdot n$ for any $c \geq 10$.)

We also drop *lower-order terms*: any term that grows more slowly than the fastest-growing term. As n gets large, the fastest-growing term comes to dominate. (The next three slides show this happening.)

Therefore: if you've got an algorithm that takes $n^2 + n + \log(n)$ steps, that's an $O(n^2)$ algorithm.

Why we do things this way



Suppose you counted exactly, and got $n^2 + n$ steps.

At small sizes, the n part counts:

At $n = 1$: n^2 is 1 and n is 1. The n part is *half* the total.

At $n = 10$: n^2 is 100 and n is 10. The n part is still about 9%.

Why we do things this way

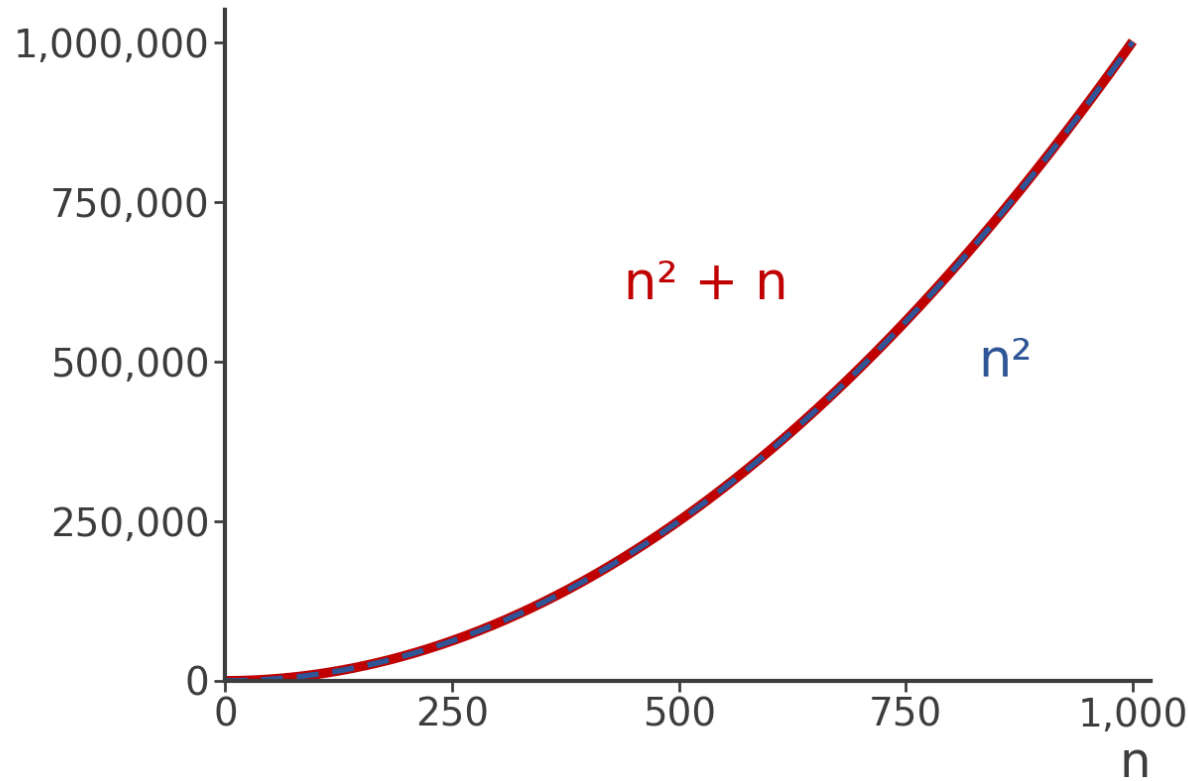
Same $n^2 + n$, at bigger and bigger n :

n	n^2 part	n part	n part's share
1	1	1	50%
10	100	10	9%
100	10,000	100	1%
1,000	1,000,000	1,000	0.1%
1,000,000	1,000,000,000,000	1,000,000	0.0001%

÷ 10

Each time n grows 10×, the n part's share of the total shrinks about 10×.

Why we do things this way



Zoom out, and the two curves become one.

At $n = 1,000,000$: the n^2 part is a trillion, and the n part is a million, one millionth of the total. It has stopped mattering.

So we keep only the fastest-growing term: $n^2 + n$ is $O(n^2)$.

Big Ω

Big $O \rightarrow$ asymptotic *upper* bound:

$f(n)$ is $O(g(n))$ iff there exist $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n > n_0$

There's a related concept, *big $\Omega \rightarrow$ asymptotic lower* bound:

$f(n)$ is $\Omega(g(n))$ iff there exist $c > 0$ and n_0 such that $f(n) \geq c \cdot g(n)$ for all $n > n_0$

In words: $f(n)$ grows at least as fast as $g(n)$.

Note: upper vs. lower bound is *not* the same thing as worst vs. best case. We can put O or Ω bounds on either one. For example, insertion sort's best case is $O(n)$, and its worst case is $\Omega(n^2)$.

Big Θ

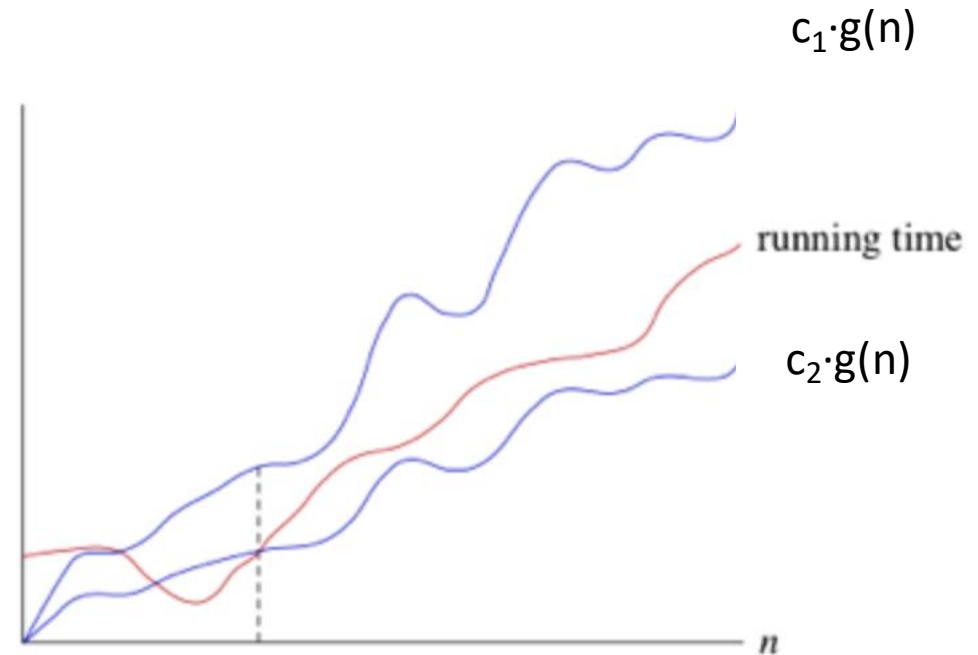
If $g(n)$ is both an upper *and* a lower bound for $f(n)$, then $f(n)$ is $\Theta(g(n))$.

More formally:

$f(n)$ is $\Theta(g(n))$ iff there exist constants c_1 , $c_2 > 0$ and n_0 such that

$c_2 \cdot g(n) \leq f(n) \leq c_1 \cdot g(n)$ for all $n > n_0$

Equivalently: $f(n)$ is both $O(g(n))$ and $\Omega(g(n))$.



~ notation

- Sedgewick (*Algorithms*, 4th ed., Sedgewick & Wayne) uses an analysis tool that is related to, but somewhat different from, Big O notation. He calls it ~ (tilde) notation.
- The chief difference between Big O and ~ is that ~ keeps the constant on the leading term.
- How to do it:
 - 1) Estimate the running time as a function of the size n of our data set.
 - 2) Discard lower-order terms (as in Big O).
 - 3) Keep the constant that multiplies the fastest-growing term.

More formally: $f(n) \sim g(n)$ iff $f(n) / g(n)$ approaches 1 as n goes to infinity.

For example, $3n^2 + 10n \sim 3n^2$, while in Big O it's just $O(n^2)$. ~ is the more precise statement: if $f(n) \sim g(n)$, then $f(n)$ is $\Theta(g(n))$.

Common features of Big O and $\sim$

- The term that matters is the *fastest-growing* term.
 - If an algorithm takes $n^2 + n \cdot \log(n) + 2$ steps, we care about the **n^2** term
 - If an algorithm takes $n \cdot \log(n) + \log(n) + 2$ steps, we care about the **$n \cdot \log(n)$** term
 - If an algorithm takes $\log(n) + 2$ steps, we care about the **$\log(n)$** term

Things can get *much* worse than polynomial time. Exponential terms (2^n) dominate polynomial terms, and factorial terms ($n!$) dominate exponential ones. If we're using an algorithm like that... we're not going to get much work done, except on small inputs.